MIT 8103 Advanced Database Systems — Portfolio Summary

Fakiyesi Vincent · Student ID 2026/A/MIT/0304 · M.Sc. Information Technology, Miva Open University

2026/2027 First Semester Continuous Assessment · GitHub repository:
https://github.com/<your-username>/advanced-database-portfolio

1. Selected organisation

NairaVault is a simulated Nigerian digital-wallet (fintech) platform: customers open a wallet with a phone number, complete KYC to unlock higher limits, fund the wallet, and transfer money to other users or to external bank accounts. The users of the database are customers (app/USSD), support staff, compliance officers, the finance/reconciliation team and scheduled system jobs. The major data are customers, accounts (wallets), an append-only transaction ledger, transfers, beneficiaries, banks, KYC documents, staff and an audit log. The key operations are opening accounts, transferring funds atomically, reversing failed transfers, auditing balances against the ledger, producing statements and detecting suspicious activity. The same case study is used in all five portfolios. All relational work was done on PostgreSQL 16; the test dataset holds 3,000 customers, 3,719 wallets, 67,245 transfers and 242,531 ledger rows.

2. The five portfolio activities

Portfolio 1 — Database design and modelling

I designed a nine-table schema in 3NF/BCNF with a deliberate, documented denormalisation: accounts.balance caches what the ledger can derive, and transactions.balance_after lets that cache be audited. Business rules live in the engine: CHECK constraints (non-negative balances, adult customers, internal-or-external transfer, no self-transfer), UNIQUE natural keys, foreign keys without cascading deletes on financial history, and a BEFORE UPDATE OR DELETE trigger that makes the ledger immutable. Eleven deliberately invalid statements were all rejected, and the cached balance of every wallet reconciled with its ledger. Evidence: schema, ER diagram (Figure 1), constraint-test output, normalisation document.

Portfolio 2 — Query processing and optimisation

Eight business queries were timed and their EXPLAIN (ANALYZE, BUFFERS) plans captured on the bare schema, then again after indexes chosen from the plans. A composite (account_id, created_at) index cut the customer-statement query from 15.8 ms to 0.3 ms (3,280 pages read → 7). A partial covering index turned the fee report into an index-only scan with zero heap fetches. A correlated subquery that scanned the transfers table over 2,000 times (11 s) dropped to 54 ms with foreign-key indexes — while the same question written as a JOIN/GROUP BY/HAVING was already 20 ms with no extra index, showing that query shape matters as much as indexing. The full-ledger reconciliation query correctly stayed a sequential scan.

Portfolio 3 — Transactions and concurrency

transfer_funds() performs the debit, fee, credit, ledger and audit writes in one PL/pgSQL transaction, locking both accounts with SELECT … FOR UPDATE in ascending id order. Single-session demos show COMMIT, ROLLBACK, automatic rollback on an insufficient-funds exception (0 ledger rows written), SAVEPOINT partial rollback, a compensating reversal, and the KYC daily limit refusing a transfer. Two-session demos on real threads show a lost update (₦4,000 vanished), the FOR UPDATE fix, a non-repeatable read under READ COMMITTED that REPEATABLE READ prevents, a deadlock detected by PostgreSQL after 1 s, and zero deadlocks for the same pattern through the ordered-locking procedure. A 16-thread stress test completed 794 transfers with no negative balance and every account reconciled.

Portfolio 4 — NoSQL and advanced data models

NairaVault was remodelled for MongoDB: one customer document with embedded wallets, beneficiaries (bank name denormalised) and KYC; one document per ledger entry with a compound (accountNumber, createdAt) index; a key-value daily-limit collection using atomic $inc upserts and a TTL index; and schema-free notifications. A mongosh script with JSON Schema validation, indexes, updates, aggregation pipelines and $lookup is provided for a real server, and a pymongo script migrates the live PostgreSQL data and runs the same queries. Because a MongoDB server could not be installed in my environment, that script runs on mongomock, which is stated openly and whose timings are not presented as MongoDB performance. The same document model was also built inside PostgreSQL with JSONB, where a GIN index turned a containment query from a 1.87 ms sequential scan into a 0.07 ms index scan. The assessment is that documents suit the read-heavy experience layer while the ledger stays relational (polyglot persistence).

Portfolio 5 — Distributed and cloud database exercise

A real streaming replica was built with pg_basebackup -R and a replication slot. pg_stat_replication showed state = streaming with ~7–30 ms replay lag; a write on the primary appeared on the replica; the replica refused an INSERT (read-only); a transfer replicated with its balances; the heavy reconciliation query was offloaded to the replica; and pg_ctl promote turned the replica into a writable primary in two seconds. The ledger was then partitioned by month (the October report was pruned to one of twelve partitions; a month of history was

detached in a millisecond) and by hash of account_id into four shards (balanced within 4%; a per-account query hit one shard, a cross-account report fanned out to all four). The report discusses asynchronous versus synchronous replication for a ledger, read-your-own-writes, CAP choices per data type, and a Multi-AZ RDS + Atlas cloud design with a note on Nigerian data residency.

3. Evidence

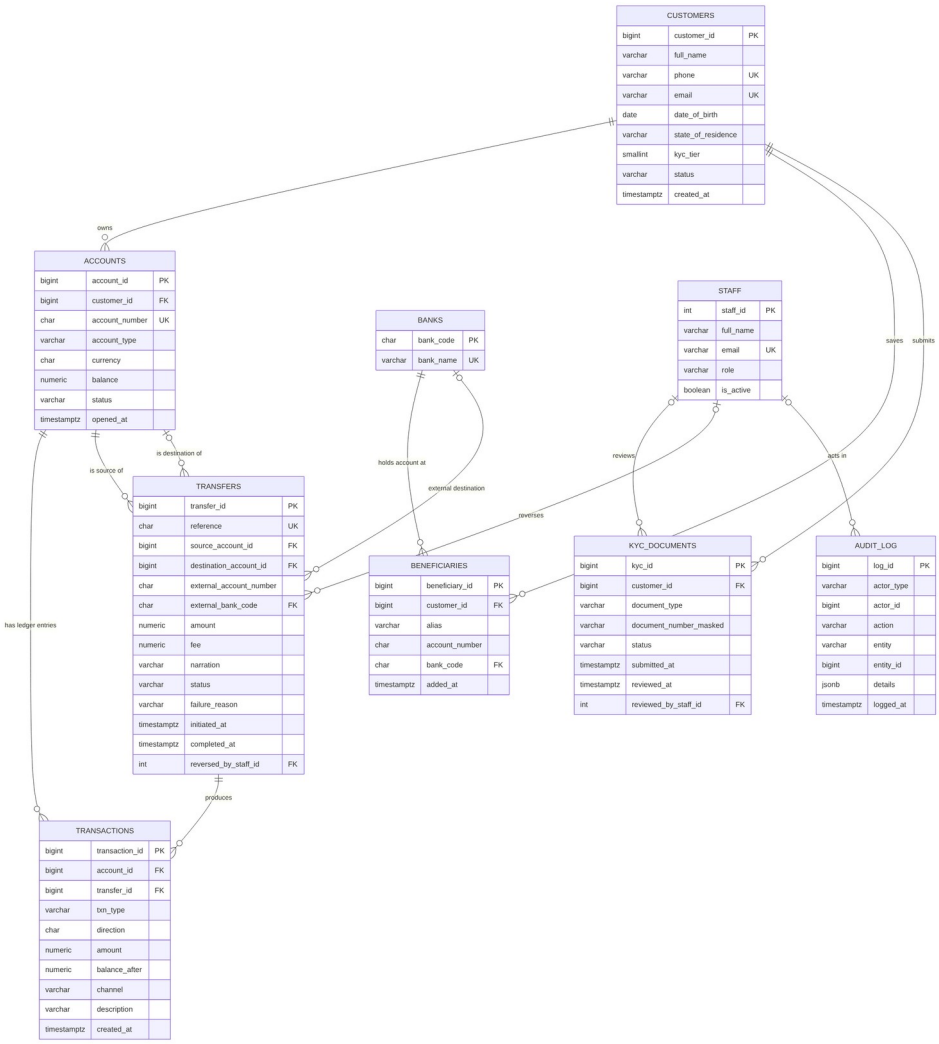


Figure 1. NairaVault entity-relationship diagram (Portfolio 1).

Selected measurements captured from the evidence files in the repository:

Portfolio	Measurement	Result
1	Invalid statements rejected / balances reconciled	11 of 11 / 3,719 of 3,719
2	Statement query, before → after composite index	15.8 ms → 0.3 ms (3,280 → 7 pages)
2	Correlated subquery, before → after FK indexes	11,070 ms → 54 ms
3	Naive concurrent withdrawals / with FOR UPDATE	₦4,000 lost / correct
3	Stress test: 794 concurrent transfers	0 negative balances, 0 ledger mismatches
4	JSONB containment query, Seq Scan → GIN index	1.87 ms → 0.07 ms
5	Replication state / replay lag / bytes behind after 2,000 inserts	streaming / ~7 ms / 0
5	Failover: promote replica and write to it	pg_is_in_recovery() = f, INSERT succeeded

Every folder in the repository has a README explaining how the work was implemented and tested and an evidence/ folder with the unedited terminal output, execution plans, logs and exported documents.

4. Individual technical reflection

Design choices. The decision I am most confident about is treating the ledger as the source of truth and the balance as a cache with a running balance_after column. It let me prove consistency with one query after every

experiment, and that query caught two real bugs: a seed-data row that violated a CHECK constraint, and a sort tie-break error (a transfer's debit and fee rows share a timestamp) that showed up only in the document-model check. I also chose to put business rules — limits, status checks, funds checks — inside the database transaction rather than in application code, so that the check and the write are covered by the same lock.

Challenges. The hardest part was concurrency. Seeing a withdrawal silently disappear in the lost-update demo made the theory concrete, and the deadlock log — both sessions 'waiting' and then PostgreSQL killing one exactly one second later — explained why the transfer procedure locks accounts in a fixed order. The second challenge was environmental: I could not install a MongoDB server, so I had to decide between faking evidence and being transparent; I chose to run the real driver code on a compatible test engine, say so clearly, and back it with a JSONB implementation on a real engine.

Lessons learned. Indexes are not magic: the reconciliation query got no faster because it must read every row, and the correlated subquery was better fixed by rewriting it. Distribution is a trade, not a free upgrade: hash partitioning made the per-account query faster and the cross-account report slower, which is exactly why the shard key must follow the dominant access pattern. Finally, building replication by hand made managed cloud services make sense — Multi-AZ is a synchronous standby plus the failover I ran manually, with monitoring and fencing around it.

5. References and AI-use declaration

Full references are in final-documentation/Fakiyesi_Vincent_References.md (Elmasri & Navathe 2017; Silberschatz, Korth & Sudarshan 2019; Kleppmann 2017; PostgreSQL 16 and MongoDB documentation; CBN mobile-money framework; NDPA 2023; AWS RDS documentation).

AI use: Claude (Anthropic) was used to draft scripts, documentation and explanations from my case-study decisions, to run the scripts in a sandbox so that the evidence is genuine output, and to help debug failures. I reviewed every analysis against the evidence files, corrected and re-ran scripts where needed, and can explain the design decisions in my own words. The full declaration is in final-documentation/Fakiyesi_Vincent_AIUseDeclaration.md. I take full responsibility for the accuracy and originality of this submission.